

🍷 Mac Mini M4 Pro — AWS Deployment Guide

Deploy Final Evolution Lab from your Mac Mini M4 Pro to AWS

Monthly Budget: ~\$400/month

Total Deployment Time: ~30-60 minutes (after UE5 build is complete)

Prerequisites: Completed [MAC_MINI_SETUP_GUIDE.md](#) with a successful Linux build

Table of Contents

1. [Install Required Tools](#)
 2. [Configure AWS Credentials](#)
 3. [Set Up Billing Alerts](#)
 4. [Deploy Infrastructure with Terraform](#)
 5. [Upload Build to S3](#)
 6. [Deploy to EC2](#)
 7. [Verify Deployment](#)
 8. [Monitoring from Mac](#)
 9. [Cost Management](#)
 10. [Teardown / Stop Resources](#)
-

1. Install Required Tools

🕒 **Time: 10-15 minutes**

Install Homebrew (if not already installed)

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

```
# Add Homebrew to PATH (Apple Silicon Macs)
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

Install AWS CLI

```
brew install awscli
```

```
# Verify installation
aws --version
# Expected: aws-cli/2.x.x Python/3.x.x Darwin/... source/arm64
```

Install Terraform

```
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
```

```
# Verify installation
terraform --version
# Expected: Terraform v1.x.x on darwin_arm64
```

Install Additional Tools

```
# jq for JSON parsing
brew install jq
```

```
# Session Manager Plugin (for SSH to EC2 without key pairs)
brew install --cask session-manager-plugin
```

```
# Watch command for monitoring
brew install watch
```

2. Configure AWS Credentials

🕒 **Time: 10 minutes**

Step 2.1: Create an IAM User

1. Log in to [AWS Console](#)
2. Go to **IAM** → **Users** → **Create User**
3. Username: fel-deployer
4. Select **“Attach policies directly”**
5. Attach these policies:
 - AmazonEC2FullAccess
 - AmazonS3FullAccess
 - AmazonVPCFullAccess
 - IAMFullAccess
 - CloudWatchFullAccess
 - ElasticLoadBalancingFullAccess
6. Click **Create User**
7. Go to the user → **Security credentials** → **Create access key**
8. Choose **“Command Line Interface (CLI)”**
9. **Save the Access Key ID and Secret Access Key** — you’ll need

them next

Step 2.2: Configure AWS CLI

aws configure

Enter when prompted:

```
AWS Access Key ID: AKIA.....
AWS Secret Access Key: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
Default region name: us-west-2
Default output format: json
```

🔔 **Region choice:** us-west-2 (Oregon) is recommended for West Coast users. It has good GPU instance availability and competitive pricing.

Step 2.3: Verify Credentials

Should return your account info

```
aws sts get-caller-identity
```

Expected output:

```
# {
#   "UserId": "AIDA...",
#   "Account": "123456789012",
#   "Arn": "arn:aws:iam::123456789012:user/fel-deployer"
# }
```

3. Set Up Billing Alerts

🕒 **Time: 5 minutes**

💡 **DO THIS BEFORE deploying anything!** GPU instances cost ~\$12-18/day.

Create a \$400/month budget with email alerts

```
aws budgets create-budget \
  --account-id $(aws sts get-caller-identity --query Account --
    output text) \
  --budget '{
    "BudgetName": "FEL-Monthly-400",
    "BudgetLimit": {"Amount": "400", "Unit": "USD"},
    "TimeUnit": "MONTHLY",
    "BudgetType": "COST"
  }' \
  --notifications-with-subscribers '[
    {
```

```

        "Notification": {"NotificationType": "ACTUAL",
            "ComparisonOperator": "GREATER_THAN", "Threshold": 50},
        "Subscribers": [{"SubscriptionType": "EMAIL", "Address":
            "YOUR_EMAIL@example.com"}]
    },
    {
        "Notification": {"NotificationType": "ACTUAL",
            "ComparisonOperator": "GREATER_THAN", "Threshold": 80},
        "Subscribers": [{"SubscriptionType": "EMAIL", "Address":
            "YOUR_EMAIL@example.com"}]
    },
    {
        "Notification": {"NotificationType": "ACTUAL",
            "ComparisonOperator": "GREATER_THAN", "Threshold": 100},
        "Subscribers": [{"SubscriptionType": "EMAIL", "Address":
            "YOUR_EMAIL@example.com"}]
    }
]'
```

Replace YOUR_EMAIL@example.com with your actual email.

4. Deploy Infrastructure with Terraform

🕒 Time: 10-15 minutes

Step 4.1: Navigate to Terraform Directory

```
cd ~/Projects/rork-final-evolution-lab/deployment/aws
```

Step 4.2: Configure Terraform Variables

```
# Copy the example variables file
cp terraform.tfvars.example terraform.tfvars

# Edit the variables (use your preferred editor)
nano terraform.tfvars
# OR
open -e terraform.tfvars
```

Update terraform.tfvars:

```
# AWS Region
aws_region = "us-west-2"

# Your project name
project_name = "final-evolution-lab"

# Environment
environment = "production"
```

```
# EC2 Instance Type for GPU server
# Budget option (~$12/day): g4dn.xlarge
# Better option (~$18/day): g5.xlarge
gpu_instance_type = "g4dn.xlarge"

# SSH Key name (we'll create this)
key_name = "fel-mac-mini"

# Your IP for SSH access (find with: curl ifconfig.me)
allowed_ssh_cidr = "YOUR_IP/32"

# Notification email
notification_email = "YOUR_EMAIL@example.com"

# Domain (optional)
# domain_name = "stream.finalevolutiongroup.com"
```

Step 4.3: Create SSH Key Pair

```
# Generate SSH key pair
ssh-keygen -t ed25519 -f ~/.ssh/fel-mac-mini -C "fel-deploy@mac-mini" -N ""

# Import to AWS
aws ec2 import-key-pair \
  --key-name "fel-mac-mini" \
  --public-key-material fileb://~/.ssh/fel-mac-mini.pub \
  --region us-west-2

# Verify
aws ec2 describe-key-pairs --key-names "fel-mac-mini" --region us-west-2
```

Step 4.4: Initialize and Apply Terraform

```
cd ~/Projects/rork-final-evolution-lab/deployment/aws
```

```
# Initialize Terraform
```

```
terraform init
```

```
# Preview what will be created
```

```
terraform plan
```

```
# Review the plan carefully, then apply
```

```
terraform apply
```

When prompted Do you want to perform these actions?, type **yes** and press Enter.

Step 4.5: Save Terraform Outputs

```
# Save outputs for later use
terraform output -json > ~/Projects/rork-final-evolution-
lab/terraform-outputs.json
```

```
# View key outputs
terraform output
```

Note these values: - **S3 bucket name** — for uploading builds - **EC2 instance ID** — for SSH access - **ALB DNS name** — for accessing the game

5. Upload Build to S3

🕒 **Time: 10-30 minutes** (depends on build size and upload speed)

Step 5.1: Verify Build Exists

```
# Check build directory
ls ~/Projects/rork-final-evolution-lab/Builds/Linux/

# Check size
du -sh ~/Projects/rork-final-evolution-lab/Builds/Linux/
```

Step 5.2: Create Tarball

```
cd ~/Projects/rork-final-evolution-lab/Builds/

# Compress the build
tar -czf fel-linux-build.tar.gz Linux/

# Check compressed size
ls -lh fel-linux-build.tar.gz
```

Step 5.3: Upload to S3

```
# Get bucket name from Terraform output
BUCKET=$(cd ~/Projects/rork-final-evolution-lab/deployment/aws &&
terraform output -raw s3_bucket_name 2>/dev/null || echo
"fel-builds-bucket")

# Upload with progress
aws s3 cp ~/Projects/rork-final-evolution-lab/Builds/fel-linux-
build.tar.gz \
s3://$BUCKET/builds/fel-linux-build.tar.gz \
--region us-west-2
```

```
# Verify upload
```

```
aws s3 ls s3://$BUCKET/builds/
```

💡 **For slow connections:** Use `aws s3 cp` with `--expected-size` for large files, or use multipart upload:

```
# Multipart upload for files > 5 GB
```

```
aws s3 cp ~/Projects/rork-final-evolution-lab/Builds/fel-linux-  
build.tar.gz \  
s3://$BUCKET/builds/fel-linux-build.tar.gz \  
--region us-west-2 \  
--expected-size ${stat -f%z ~/Projects/rork-final-evolution-  
lab/Builds/fel-linux-build.tar.gz}
```

6. Deploy to EC2

🕒 **Time: 10-15 minutes**

Step 6.1: SSH into EC2 Instance

```
# Get instance public IP
```

```
INSTANCE_IP=$(aws ec2 describe-instances \  
--filters "Name=tag:Project,Values=final-evolution-lab" \  
"Name=instance-state-name,Values=running" \  
--query 'Reservations[0].Instances[0].PublicIpAddress' \  
--output text \  
--region us-west-2)
```

```
echo "Instance IP: $INSTANCE_IP"
```

```
# SSH in
```

```
ssh -i ~/.ssh/fel-mac-mini ubuntu@$INSTANCE_IP
```

Step 6.2: Deploy on EC2 (Run These Commands on EC2)

```
# On EC2 instance:
```

```
# Download build from S3
```

```
aws s3 cp s3://YOUR_BUCKET/builds/fel-linux-build.tar.gz /opt/fel/
```

```
# Extract
```

```
cd /opt/fel
```

```
tar -xzf fel-linux-build.tar.gz
```

```
# Make scripts executable
```

```
chmod +x Linux/BasketballGame/Binaries/Linux/BasketballGameServer
```

```
chmod +x Linux/*.sh 2>/dev/null
```

```
# Start the game server with Pixel Streaming
./Linux/BasketballGame/Binaries/Linux/BasketballGameServer \
  -RenderOffScreen \
  -PixelStreamingIP=0.0.0.0 \
  -PixelStreamingPort=8888 \
  -Res=1920x1080 &
```

Step 6.3: Alternative — Use Deploy Script from Mac

```
# From your Mac Terminal:
cd ~/Projects/rork-final-evolution-lab

# Use the Mac deploy script (one command)
bash deployment/scripts/mac-deploy.sh
```

See [deployment/scripts/mac-deploy.sh](#) for the full automated deployment.

7. Verify Deployment

🕒 **Time: 5 minutes**

```
# From your Mac Terminal:

# Check EC2 instance is running
aws ec2 describe-instances \
  --filters "Name=tag:Project,Values=final-evolution-lab" \
  --query 'Reservations[].Instances[].[ID:InstanceId,State:State.Name,IP:PublicIpAddress]' \
  --output table \
  --region us-west-2

# Test signalling server
curl -s http://$INSTANCE_IP:8888/healthz

# Test web frontend
curl -s -o /dev/null -w "%{http_code}" http://$INSTANCE_IP:80

# Open in browser
open http://$INSTANCE_IP:80
```

8. Monitoring from Mac

AWS Console (Web)

```
# Open AWS Console in browser
open https://us-west-2.console.aws.amazon.com/ec2/home?region=us-west-2
```

Terminal Monitoring

```
# Watch instance status (updates every 30s)
watch -n 30 'aws ec2 describe-instances \
  --filters "Name=tag:Project,Values=final-evolution-lab" \
  --query "Reservations[].Instances[0].\
    {State:State.Name,Type:InstanceType,IP:PublicIpAddress}" \
  --output table --region us-west-2'

# Check CloudWatch CPU metrics
aws cloudwatch get-metric-statistics \
  --namespace AWS/EC2 \
  --metric-name CPUUtilization \
  --dimensions Name=InstanceId,Value=YOUR_INSTANCE_ID \
  --start-time $(date -u -v -1H +%Y-%m-%dT%H:%M:%S) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%S) \
  --period 300 \
  --statistics Average \
  --region us-west-2

# View CloudWatch logs
aws logs tail /aws/ec2/fel-server --follow --region us-west-2
```

GPU Monitoring (via SSH)

```
# SSH in and check GPU
ssh -i ~/.ssh/fel-mac-mini ubuntu@$INSTANCE_IP 'nvidia-smi'

# Continuous GPU monitoring
ssh -i ~/.ssh/fel-mac-mini ubuntu@$INSTANCE_IP 'watch -n 2 nvidia-smi'
```

Quick Health Check Script

```
# Save this as ~/Projects/rork-final-evolution-lab/check-health.sh
cat > ~/Projects/rork-final-evolution-lab/check-health.sh <<
'SCRIPT'

#!/bin/bash
IP=$(aws ec2 describe-instances \
  --filters "Name=tag:Project,Values=final-evolution-lab"
  "Name=instance-state-name,Values=running" \
  --query 'Reservations[0].Instances[0].PublicIpAddress' \
  --output text --region us-west-2)

echo "📌 Instance IP: $IP"
```

```

echo "📡 Signalling: $(curl -s -o /dev/null -w '%{http_code}'
http://$IP:8888/healthz)"
echo "🌐 Frontend: $(curl -s -o /dev/null -w '%{http_code}'
http://$IP:80)"
echo "🖥️ GPU Status:"
ssh -i ~/.ssh/fel-mac-mini ubuntu@$IP 'nvidia-smi --query-
gpu=utilization.gpu,memory.used,memory.total --
format=csv,noheader' 2>/dev/null || echo " (SSH not
available)"

SCRIPT

chmod +x ~/Projects/rork-final-evolution-lab/check-health.sh

```

9. Cost Management

Daily Cost Breakdown (\$400/month budget)

Resource	Instance	Monthly Cost	Daily Cost
GPU Server	g4dn.xlarge	~\$180-360	~\$6-12
S3 Storage	10 GB	~\$0.23	~\$0.01
Data Transfer	~100 GB/mo	~\$9	~\$0.30
ALB	1	~\$16	~\$0.53
CloudWatch	Basic	~\$0	\$0
Total		~\$205-385	~\$7-13

Start/Stop Instances to Save Money

```

# 🛑 Stop instance when not in use (saves ~$12/day)
aws ec2 stop-instances \
  --instance-ids YOUR_INSTANCE_ID \
  --region us-west-2

# ▶ Start instance when needed
aws ec2 start-instances \
  --instance-ids YOUR_INSTANCE_ID \
  --region us-west-2

# 🔄 Get new public IP after restart
aws ec2 describe-instances \
  --instance-ids YOUR_INSTANCE_ID \
  --query 'Reservations[0].Instances[0].PublicIpAddress' \
  --output text --region us-west-2

```

💡 **Pro Tip:** Stop instances overnight and on days you're not testing. This can cut costs by 50-70%.

Schedule Auto-Stop

```
# Create a Lambda function to auto-stop at midnight
# Or use a simple cron approach via Mac:

# Add to your Mac's crontab (stops EC2 at midnight PST)
crontab -e
# Add this line:
# 0 0 * * * aws ec2 stop-instances --instance-ids YOUR_INSTANCE_ID -
    -region us-west-2
```

10. Teardown / Stop Resources

Stop Everything (Keep Infrastructure)

```
# Stop EC2 instances
aws ec2 stop-instances \
  --instance-ids $(aws ec2 describe-instances \
    --filters "Name=tag:Project,Values=final-evolution-lab" \
    --query 'Reservations[].Instances[].InstanceId' \
    --output text --region us-west-2) \
  --region us-west-2
```

Full Teardown (Destroy Everything)

⚠ **This deletes ALL AWS resources.** Only do this if you're done with the project.

```
cd ~/Projects/rork-final-evolution-lab/deployment/aws
```

```
# Preview what will be destroyed
terraform plan -destroy
```

```
# Destroy all resources
terraform destroy
```

When prompted, type **yes** to confirm.

Mac-Specific Terminal Tips

Useful Aliases

Add these to ~/.zshrc for quick access:

```
# Add to ~/.zshrc
cat >> ~/.zshrc << 'EOF'

# Final Evolution Lab shortcuts
```

```
export FEL_DIR="$HOME/Projects/rork-final-evolution-lab"
alias fel="cd $FEL_DIR"
alias fel-status="bash $FEL_DIR/check-health.sh"
alias fel-deploy="bash $FEL_DIR/deployment/scripts/mac-deploy.sh"
alias fel-ssh="ssh -i ~/.ssh/fel-mac-mini ubuntu@$(aws ec2
    describe-instances --filters 'Name=tag:Project,Values=final-
    evolution-lab' 'Name=instance-state-name,Values=running' --
    query 'Reservations[0].Instances[0].PublicIpAddress' --
    output text --region us-west-2)"
alias fel-stop="aws ec2 stop-instances --instance-ids $(aws ec2
    describe-instances --filters 'Name=tag:Project,Values=final-
    evolution-lab' --query
    'Reservations[].Instances[].InstanceId' --output text --
    region us-west-2) --region us-west-2"
alias fel-start="aws ec2 start-instances --instance-ids $(aws ec2
    describe-instances --filters 'Name=tag:Project,Values=final-
    evolution-lab' --query
    'Reservations[].Instances[].InstanceId' --output text --
    region us-west-2) --region us-west-2"
```

E0F

```
source ~/.zshrc
```

iTerm2 Profile (Optional)

If using iTerm2, create a profile with: - Name: FEL Deploy - Working
Directory: ~/Projects/rork-final-evolution-lab - Send text at start:
source .ue5_env 2>/dev/null

Last updated: April 2, 2026

Optimized for Mac Mini M4 Pro deploying to AWS